Neovim Verilog / SystemVerilog Setup

Reference for the Verible-based tooling added to your LazyVim config (`~/.config/nvim/lua/plugins/verilog.lua` and `init.lua`).

What's installed

- **Treesitter** — syntax highlighting + indent (grammar: `systemverilog`, covers both filetypes)
- **verible-verilog-ls** — LSP: diagnostics, go-to-definition, hover, etc.
- **verible-verilog-lint** — style linting via nvim-lint, runs on save
- **verible-verilog-format** — auto-formatting via conform.nvim, runs on save
- **<leader>vr** — compile + run the current file's testbench with iverilog/vvp

Filetype detection

Pinned explicitly in `init.lua` so it can never be guessed wrong:

Extension	Filetype
.v	verilog
.sv	systemverilog
.svh	systemverilog

> Without this, Neovim's built-in `.v` detection is a content heuristic that disambiguates against Coq and the V language — it can guess wrong on an empty or comment-only file.

LSP (verible-verilog-ls)

Nothing to run — it attaches automatically when you open a .v/.sv/.svh file.

- `:LspInfo` — confirm the `verible` client is attached
- `gd` / `grr` — go to definition / references (LazyVim defaults)
- `K` — hover documentation

> Three other mason-installed servers that also claim these filetypes (hdl_checker, slang_server, svlangserver) are explicitly disabled — hdl_checker crashed on attach and the others just added noisy duplicate diagnostics.

Linting (verible-verilog-lint)

Runs automatically on `:w` (save). Issues show up as normal diagnostics — underlines in the buffer, and in the signs column.

- `:w` — save, lint runs, diagnostics update
- `]d` / `[d` — jump to next/previous diagnostic (LazyVim defaults)
- `<leader>cd` or `:lua vim.diagnostic.open_float()` — view the message under the cursor

Formatting (verible-verilog-format)

Runs automatically on `:w` for .v/.sv/.svh buffers only. Your global `vim.g.autoformat = false` setting is left untouched for every other filetype — Verilog/SystemVerilog buffers get a per-buffer opt-in instead.

- `:w` — save, buffer is reformatted first

- `<leader>uf` — toggle autoformat for the current buffer (LazyVim default), if you ever want to disable it temporarily

Compile & run: <leader>vr

Works from either the design file or its testbench — it finds the matching sibling automatically by filename convention.

- On `counter.v` → looks for `counter_tb.v` next to it
- On `counter_tb.v` → looks for `counter.v` next to it

Both files must exist and share a basename (`name.v` / `name_tb.v`). It compiles with `iverilog -g2012` and runs the result with `vvp`, opening a scratch buffer in a vertical split with the output (your `$display` lines, or the compiler error if it failed to build).

Example session:

```
$ nvim ~/dev/SystemVerilogPractise/mux2.v
(edit mux2.v and mux2_tb.v as usual)
<leader>v r -- press leader, then v, then r
-> vertical split opens:
=== vvp output (mux2_tb.v) ===
sel=0 a=0 b=1 y=0
sel=1 a=0 b=1 y=1
TEST_DONE
```

The output buffer is scratch (not written to disk) — close it with `:q` or `<C-w>q` like any split.

Files changed

- `~/.config/nvim/init.lua` — added the `vim.filetype.add(...)` block
- `~/.config/nvim/lua/plugins/verilog.lua` — new file: all of the above

Nothing else in your config was modified. All tooling (iverilog, vvp, gtkwave, verible) was already installed on this system.